



Video -> CSS

INTERACCION EN LA WEB

DESCRIPCIÓN BREVE

Gestión Multimedia en HTML5: Imágenes Responsivas, Audio y Vídeo Avanzado

PabloSG

2ºDAW-DIW

Índice

1.Introducción.....	3
2. Multimedia nativa en HTML5	3
3. Imágenes Responsivas y Manipulación Visual.....	3
4. APIs Multimedia y Control mediante JavaScript.....	4
5. Diseño Responsivo y Estilización con CSS3	4
6. Integración de Todo en un Proyecto Práctico	4
EJERCICIO PRÁCTICO.....	5
1. Gestión de Audio en HTML5	5
1.1 Audio con controles nativos	5
1.2 Audio con controles personalizados.....	5
2. Gestión de Vídeo con Controles Personalizados	6
2.1 Vídeo con controles nativos	6
2.2 Vídeo con controles personalizados	6
3. Gestión de Imágenes	6
3.1 Imagen estática.....	6
3.2 Imagen modificable	6
4. Controles personalizados del audio.....	7
4.1 Play / Pause.....	7
4.2 Loop	7
4.3 Velocidad	7
4.4 Descarga.....	7
5. Controles personalizados del vídeo	8
5.1 Play / Pause.....	8
5.2 Picture-in-Picture	8
5.3 Mute	8
5.4 Fullscreen.....	8
5.5 Tiempo.....	8
6. Controles personalizados de la imagen	9
6.1 Zoom.....	9
6.2 Opacidad.....	9
6.3 Rotar	9

6.4 Reset 9

1.Introducción

Este proyecto tiene como objetivo demostrar el uso de las capacidades multimedia de HTML5 mediante la integración de **audio, vídeo e imágenes, combinando HTML5, CSS3 y JavaScript** para crear una experiencia interactiva y visualmente atractiva.

Se trabajan conceptos clave como:

- Uso correcto de etiquetas multimedia modernas
- Creación de controles personalizados
- Manipulación de elementos multimedia mediante JavaScript
- Diseño responsivo y estilizado con CSS

2. Multimedia nativa en HTML5

HTML5 introduce las etiquetas **<audio>** y **<video>**, que permiten reproducir contenido multimedia sin necesidad de software adicional. Estas etiquetas incluyen atributos y APIs que facilitan su control desde JavaScript, permitiendo crear reproductores personalizados y experiencias interactivas.

Ventajas principales:

- Compatibilidad con todos los navegadores modernos
- Mejor rendimiento y menor consumo de recursos
- Control total mediante JavaScript
- Integración con CSS para personalización visual
- Soporte para subtítulos, pistas alternativas y accesibilidad

3. Imágenes Responsivas y Manipulación Visual

Las imágenes son un elemento clave en cualquier proyecto web. HTML5 y CSS3 permiten no solo mostrarlas, sino también **adaptarlas al dispositivo y modificarlas dinámicamente** mediante transformaciones y filtros.

Conceptos clave:

- **Responsividad:** adaptar el tamaño y resolución según el dispositivo
- **Filtros CSS:** efectos como blur, grayscale, saturación
- **Transformaciones:** rotación, escalado, desplazamiento
- **Interactividad:** control de efectos mediante JavaScript

Esto permite crear interfaces más dinámicas y visualmente atractivas.

4. APIs Multimedia y Control mediante JavaScript

Tanto <audio> como <video> incluyen una API muy completa que permite:

- Reproducir, pausar o reiniciar
- Controlar volumen y mute
- Detectar eventos (play, pause, timeupdate...)
- Modificar la posición de reproducción
- Activar modos avanzados como **Picture-in-Picture o Fullscreen**

Gracias a esto, es posible sustituir los controles nativos por controles personalizados, mejor integrados con el diseño del sitio.

5. Diseño Responsivo y Estilización con CSS3

CSS3 aporta herramientas esenciales para que el contenido multimedia se adapte a cualquier pantalla:

- **Flexbox y Grid** para organizar elementos
- **Media queries** para ajustar estilos según el dispositivo
- **Sombras, bordes, degradados y animaciones** para mejorar la estética
- **Transiciones** para suavizar cambios visuales

El resultado es una experiencia moderna, fluida y coherente en móviles, tablets y ordenadores.

6. Integración de Todo en un Proyecto Práctico

Un proyecto multimedia en HTML5 combina:

- **HTML5** para la estructura y los elementos multimedia
- **CSS3** para el diseño y los efectos visuales
- **JavaScript** para la interactividad y el control avanzado

Este enfoque permite crear aplicaciones web ricas, dinámicas y totalmente personalizadas, sin depender de tecnologías externas.

EJERCICIO PRÁCTICO

Configura una barra de progreso para un elemento multimedia (audio o video).

```
function setupMediaProgress(media, progressContainer, progressBar,
currentEl, durationEl) {
  media.addEventListener("loadedmetadata", () => {
    durationEl.textContent = formatTime(media.duration);
  });
  media.addEventListener("timeupdate", () => {
    currentEl.textContent = formatTime(media.currentTime);
    const percent = (media.currentTime / media.duration) * 100;
    progressBar.style.width = percent + "%";
  });
  progressContainer.addEventListener("click", (e) => {
    const rect = progressContainer.getBoundingClientRect();
    const clickX = e.clientX - rect.left;
    media.currentTime = (clickX / rect.width) * media.duration;
  });
}
```

1. Gestión de Audio en HTML5

Se implementan dos audios

1.1 Audio con controles nativos

El proyecto incluye un reproductor de audio básico con controles nativos del navegador.

```
<audio controls preload="none">
  <source src="sonido/audio.mp3" type="audio/mpeg">
  Tu navegador no soporta audio HTML5.
</audio>
```

Características:

- **controls:** muestra botones de reproducción, pausa y volumen.
- **preload="none":** mejora el rendimiento evitando la carga automática del audio.
- **source:** permite compatibilidad con distintos formatos si fuese necesario.

1.2 Audio con controles personalizados

El segundo audio se gestiona completamente mediante JavaScript, ocultando los controles nativos.

```
<audio id="audio2" preload="metadata">
  <source src="sonido/audio1.mp3" type="audio/mpeg">
</audio>
```

2. Gestión de Vídeo con Controles Personalizados

Se implementan dos vídeos:

2.1 Vídeo con controles nativos

Un vídeo estándar con controles por defecto del navegador para comparar con la versión

Características:

- Controles estándar del navegador.
- Reproducción, pausa, volumen y barra de progreso automática.
- Implementación sencilla.personalizada

```
<video controls width="100%">
  <source src="sonido/video.mp4" type="video/mp4">
</video>
```

2.2 Vídeo con controles personalizados

El segundo vídeo se gestiona completamente mediante JavaScript, ocultando los controles nativos.

```
<video id="video" preload="metadata">
  <source src="sonido/video.mp4" type="video/mp4">
</video>
```

3. Gestión de Imágenes

Se muestran dos tipos de imágenes:

3.1 Imagen estática

Una imagen sin modificaciones para comparación visual.

Características:

- No tiene efectos aplicados.
- Sirve como referencia visual.
- Se adapta al tamaño del contenedor.

```
<div class="col">
  
</div>
```

3.2 Imagen modificable

Imagen interactiva que permite aplicar distintos efectos:

```
<div class="img-wrapper">
  
</div>
```

4. Controles personalizados del audio

```
<div class="controls">
  <button id="audioPlay">▶</button>
  <label>Velocidad
  <input type="range" id="speed" min="0.5" max="2" step="0.1" value="1">
  </label>
  <button id="loopBtn">🔄 Loop</button>
  <div class="time">
    <span id="audioCurrent">0:00</span> /
    <span id="audioDuration">0:00</span>
  </div>
  <div class="progress" id="audioProgress">
    <div class="progress-bar" id="audioProgressBar"></div>
  </div>
</div>
```

4.1 Play / Pause

El botón de reproducción permite iniciar o detener el audio.

```
audioPlayBtn.addEventListener("click", () => {
  audio2.paused ? audio2.play() : audio2.pause();
});
```

4.2 Loop

El loop hace que el audio se repita automáticamente cuando llega al final.

Si está activado, el audio vuelve a empezar sin que tengas que pulsar nada.

```
<button id="loopBtn">🔄 Loop</button>
```

4.3 Velocidad

Este control ajusta la velocidad de reproducción del audio.

- Permite reproducir el audio más lento o más rápido.
- El rango definido en el HTML (0.5 a 2) permite:
 - 0.5 → mitad de velocidad
 - 1 → velocidad normal
 - 2 → el doble de velocidad

```
speedControl.addEventListener("input", () => {
  audio2.playbackRate = speedControl.value;
});
```

4.4 Descarga

El reproductor incluye un botón que permite descargar el archivo de audio directamente desde el navegador.

```
const downloadAudioBtn = document.getElementById("downloadAudio");
audio2.addEventListener("loadedmetadata", () => {
  downloadAudioBtn.href = audio2.currentSrc;
});
```

5. Controles personalizados del vídeo

```
<div class="controls">
  <button id="play"><img alt="play icon" data-bbox="360 130 375 145"/></button>
  <button id="mute"><img alt="mute icon" data-bbox="355 148 370 163"/></button>
  <button id="pip"><img alt="PiP icon" data-bbox="345 165 365 180"/> PiP</button>
  <button id="fullscreen"><img alt="fullscreen icon" data-bbox="355 183 370 198"/></button>
  <div class="time">
    <span id="current">0:00</span> /
    <span id="duration">0:00</span>
  </div>
  <div class="progress" id="progress">
    <div class="progress-bar" id="progress-bar"></div>
  </div>
  <label>Volumen:
    <input type="range" id="volume" min="0" max="1" step="0.05" value="1">
  </label>
```

5.1 Play / Pause

Se detecta si el vídeo está en reproducción o en pausa y se alterna su estado.

```
if (video.paused) {
  video.play();
} else {
  video.pause();
}
```

5.2 Picture-in-Picture

Se incluye el modo que permite reproducir vídeo en una ventana flotante.

```
await video.requestPictureInPicture();
```

5.3 Mute

Silencia o activa el audio del vídeo.

```
video.muted = !video.muted;
```

5.4 Fullscreen

Muestra el vídeo a pantalla completa.

```
video.requestFullscreen();
```

5.5 Tiempo

Se muestra la duración del vídeo.

```
video.addEventListener("loadedmetadata", () => {
  durationEl.textContent = formatTime(video.duration);
})
video.addEventListener("timeupdate", () => {
  currentTimeEl.textContent = formatTime(video.currentTime);
  const percent = (video.currentTime / video.duration) * 100;
  progressBar.style.width = percent + "%";
});
```

6. Controles personalizados de la imagen

Características:

- Permiten modificar la imagen en tiempo real.
- Efectos aplicados mediante transformaciones CSS.

```
<div class="controls">
  <button id="zoom">🔍 Zoom</button>
  <button id="opacity">🖼️ Opacidad</button>
  <button id="rotate">🔄 Rotar</button>
  <button id="reset">🔄 Reset</button>
</div>
```

6.1 Zoom

Una imagen que se amplia

```
document.getElementById("zoom").onclick = () => {
  scale += 0.2;
  updateImage();
};
```

6.2 Opacidad

Imagen que coje opacidad

```
document.getElementById("opacity").onclick = () => {
  opacity -= 0.1;
  if (opacity < 0.2) opacity = 1;
  updateImage();
};
```

6.3 Rotar

Imagen que se rota

```
document.getElementById("rotate").onclick = () => {
  rotation += 15;
  updateImage();
};
```

6.4 Reset

Imagen interactiva que permite aplicar distintos efectos:

```
document.getElementById("reset").onclick = () => {
  scale = 1;
  opacity = 1;
  rotation = 0;
  updateImage();
};
```

EJERCICIO PROPUESTO

Crear una pequeña página web que gestione audio, vídeo e imagen utilizando HTML5, CSS y JavaScript.

El objetivo es practicar eventos, manipulación del DOM y controles multimedia personalizados.

1. Control avanzado de audio

Implementar:

- Un botón Play / Pause
- Un control deslizante para ajustar la velocidad de reproducción
 - Rango recomendado: 0.5x → 2x

2. Vídeo con funciones extras

Usar un vídeo con controles nativos y añadir un botón extra:

- Fullscreen al pulsarlo, el vídeo debe ocupar toda la pantalla.
- Picture-in-Picture (PiP): el vídeo debe abrirse en una ventana flotante

3. Imagen editable

Añadir dos botones que modifiquen la imagen:

- Zoom: aumenta el tamaño de la imagen
- Reset: devuelve la imagen a su tamaño original